Jenkins Task -1

What is Jenkins?

Jenkins is an open-source automation server used to build, test, and deploy applications automatically.
It is widely used in CI/CD pipelines to help developers deliver software faster, reliably, and with fewer errors.

Advantages of Jenkins

1. Automation – Automates build, test, and deployment processes, reducing manual work.
2. CI/CD Support – Enables Continuous Integration and Continuous Delivery for faster releases.
3. Extensible – Thousands of plugins available for Git, Docker, AWS, Maven, Kubernetes, etc.
4. Open Source & Free – No licensing cost and strong community support.
5. Easy Integration – Integrates easily with version control tools, cloud platforms, and DevOps tools.
6. Scalable – Supports distributed builds using master–agent architecture.
7. Improves Code Quality – Detects issues early through automated testing.

One-line interview answer

Jenkins is an open-source CI/CD tool that automates software build, testing, and deployment to enable faster and reliable application delivery.

1. **Install Jenkins and run Jenkins on port number 8081.**

Steps:

Steps Followed to Change Jenkins Port to 8081

Jenkin Installation:

1. Added the Jenkins repository to the system:

2. sudo wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat-stable/jenkins.repo
3. Imported the Jenkins GPG key for package verification:
4. sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
5. Installed Jenkins using the package manager:
6. sudo dnf install jenkins -y
7. Installed Java (Amazon Corretto) as Jenkins runtime dependency:
8. sudo dnf install java-17-amazon-corretto -y
9. Started and enabled Jenkins service:
10. sudo systemctl start jenkins
11. sudo systemctl enable jenkins

Steps to change port number:

1. Verified Jenkins was running on the default port and identified that the port is controlled at **service startup**, not inside Jenkins UI files.

2. Checked the Jenkins systemd service using systemctl cat jenkins to understand how Jenkins is started.

3. Created a **systemd override file** under /etc/systemd/system/jenkins.service.d/override.conf.

4. Set the environment variable JENKINS_PORT=8081 inside the override file.

5. Reloaded systemd configuration using systemctl daemon-reload.

6. Restarted Jenkins service and verified Jenkins was listening on **port 8081** using ss -tulnp.

7. Allowed port **8081** in the EC2 security group and accessed Jenkins via browser.

✅ Why Override File Was Used Instead of Editing Service File

- The default Jenkins service file (/usr/lib/systemd/system/jenkins.service) is **managed by the package manager**.

- Editing the original service file can be **overwritten during Jenkins upgrades or OS updates**.

- Systemd overrides are the **recommended and safe approach** to customize service behavior.

- Overrides ensure **configuration persistence across reboots and updates**.

- This follows **Linux and DevOps best practices** for service management.

Installed java packages

```
[ec2-user@ip-172-31-21-117 ~]$ sudo wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat-stable/jenkins.repo
--2025-12-23 09:04:27--  https://pkg.jenkins.io/redhat-stable/jenkins.repo
Resolving pkg.jenkins.io (pkg.jenkins.io)... 146.75.34.133, 2a04:4e42:78::645
Connecting to pkg.jenkins.io (pkg.jenkins.io)|146.75.34.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 192 [application/octet-stream]
Saving to: '/etc/yum.repos.d/jenkins.repo'

/etc/yum.repos.d/jenkins.repo             100%[==========================================================================================>]     192  --.-KB/s    in 0s

2025-12-23 09:04:27 (18.5 MB/s) - '/etc/yum.repos.d/jenkins.repo' saved [192/192]

[ec2-user@ip-172-31-21-117 ~]$ sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
[ec2-user@ip-172-31-21-117 ~]$ sudo yum upgrade
Amazon Linux 2023 Kernel Livepatch repository                                                                  249 kB/s |  29 kB     00:0
Jenkins-stable                                                                                                  13 kB/s | 833  B     00:0
Jenkins-stable                                                                                                 215 kB/s | 3.1 kB     00:0
Importing GPG key 0xEF5975CA:
 Userid     : "Jenkins Project <jenkinsci-board@googlegroups.com>"
 Fingerprint: 6366 7EE7 4BBA 1F0A 08A6 9872 5BA3 1D57 EF59 75CA
 From       : https://pkg.jenkins.io/redhat-stable/repodata/repomd.xml.key
Is this ok [y/N]: y
Jenkins-stable                                                                                                 561 kB/s |  34 kB     00:0
Dependencies resolved.
Nothing to do.
Complete!
```

<u>JENKINS INSTALATION ERROR:</u>

1. When I was installing Jenkins from official documents java was not getting installed so tried with our institute documents as well still it was not installing

```
[ec2-user@ip-172-31-21-117 ~]$ sudo yum install fontconfig java-21-openjdk
Last metadata expiration check: 0:00:19 ago on Tue Dec 23 09:13:12 2025.
No match for argument: java-21-openjdk
Error: Unable to find a match: java-21-openjdk
[ec2-user@ip-172-31-21-117 ~]$ sudo wget -O /etc/yum.repos.d/jenkins.repo \
    https://pkg.jenkins.io/redhat/jenkins.repo
```

```
Last metadata expiration check: 0:01:48 ago on Tue Dec 23 09:13:12 2025.
Dependencies resolved.
Nothing to do.
Complete!
Last metadata expiration check: 0:01:49 ago on Tue Dec 23 09:13:12 2025.
No match for argument: java-21-openjdk
Error: Unable to find a match: java-21-openjdk
Last metadata expiration check: 0:01:50 ago on Tue Dec 23 09:13:12 2025.
Dependencies resolved.
```

```
Complete!
[ec2-user@ip-172-31-21-117 ~]$ java -version
-bash: java: command not found
[ec2-user@ip-172-31-21-117 ~]$ javac -version
-bash: javac: command not found
[ec2-user@ip-172-31-21-117 ~]$ sudo yum install java-17-openjdk -y
Last metadata expiration check: 0:05:11 ago on Tue Dec 23 09:13:12 2025.
No match for argument: java-17-openjdk
Error: Unable to find a match: java-17-openjdk
[ec2-user@ip-172-31-21-117 ~]$ sudo dnf install java-17-amazon-corretto -y
Last metadata expiration check: 0:06:07 ago on Tue Dec 23 09:13:12 2025.
Dependencies resolved.
```

Then I have instaleed with following command “sudo yum install java-17-openjdk -y”

Reason:

- Your OS is **Amazon Linux 2023**.
- Amazon Linux 2023 **does not ship java-17-openjdk packages** in its repositories.
- Instead, AWS provides **Amazon Corretto**, which is:
  - AWS’s **official, supported OpenJDK distribution**
  - Optimized and maintained by AWS
  - Fully compatible with OpenJDK

Why dnf (not yum)

- Amazon Linux 2023 uses **dnf as the package manager**.
- yum commands may exist for compatibility, but **dnf is the native tool**.

Why Jenkins works with Corretto

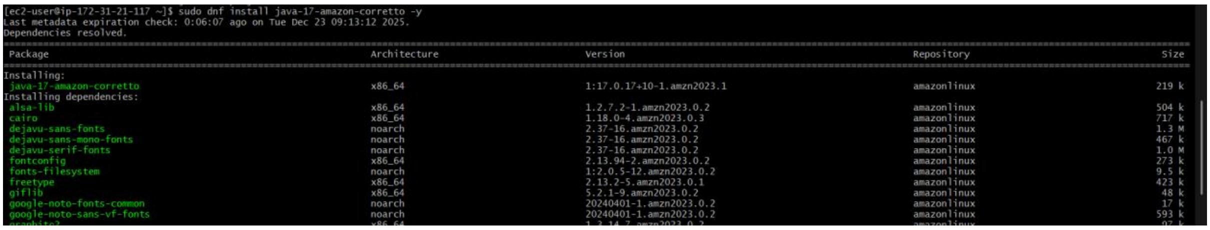
- Jenkins requires **Java 11 or 17**.
- Amazon Corretto 17 **meets Jenkins' Java requirement**.
- That's why Jenkins starts successfully once Corretto is installed.

Simple one-line explanation (for docs/interview)

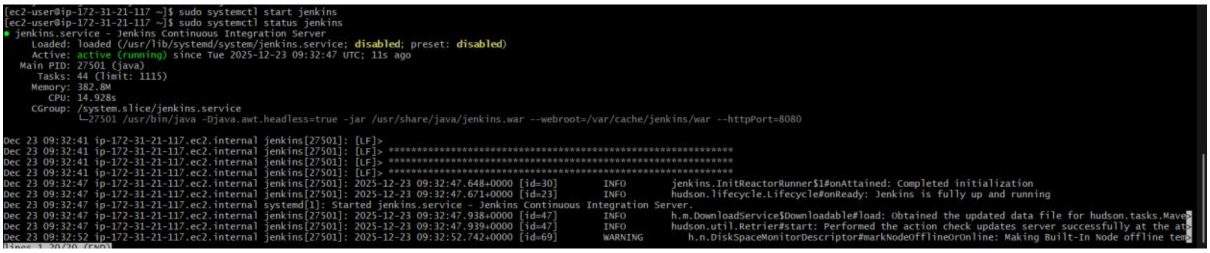
The command works because Amazon Linux 2023 uses Amazon Corretto as its supported Java distribution, which is available through dnf repositories and fully compatible with Jenkins.

If you want, I can also explain:

- Difference between **Amazon Linux 2 vs 2023**
- **Corretto vs OpenJDK**
- How to **set JAVA_HOME**
- Why Jenkins fails without Java



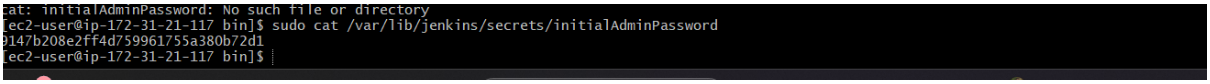
After installing Jenkins I have start and check the status Jenkins using systemctl start Jenkins and systemctl status Jenkins respectively



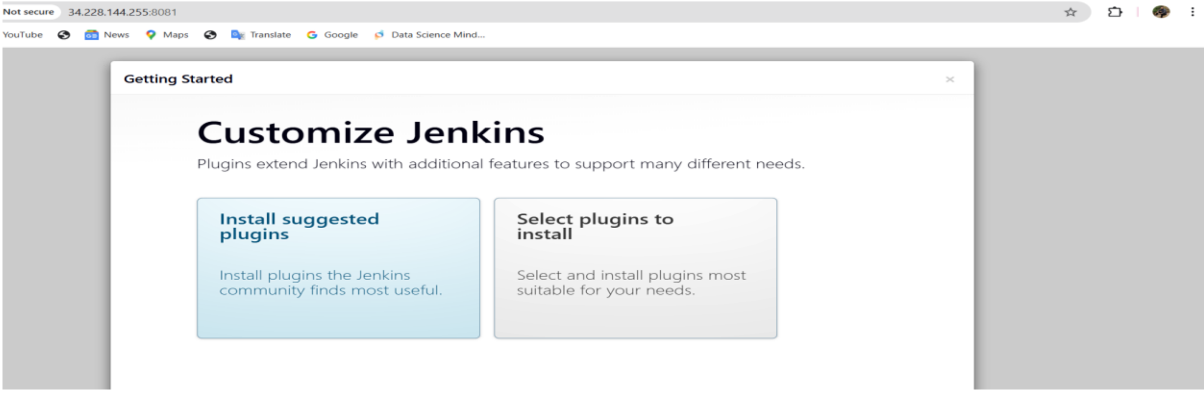
Created directory and created a file to overwrite

Jenkins port number overwrite

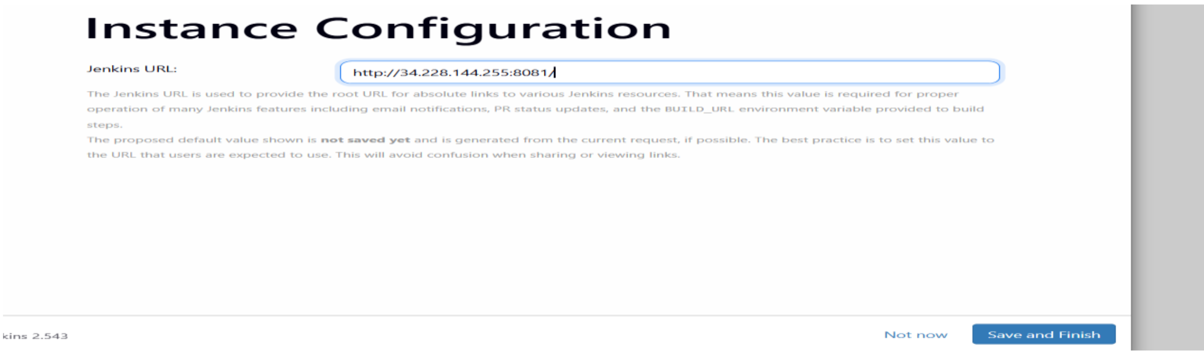
Copied password from below path



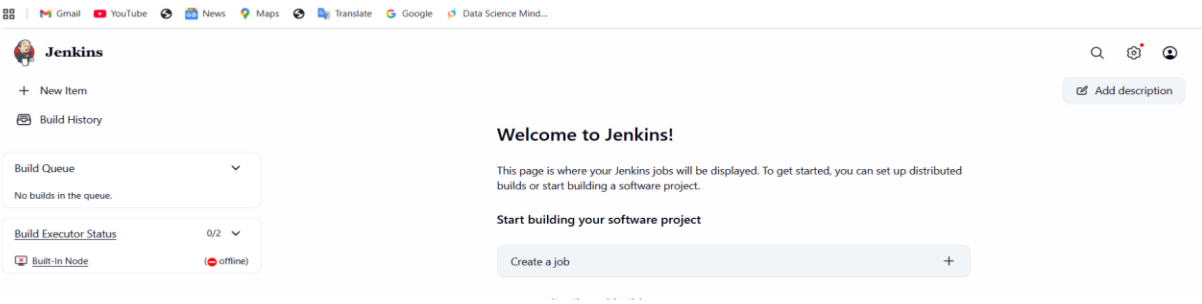
After entering password Jenkins opened, where I have selected suggested plugins



Jenkins URL



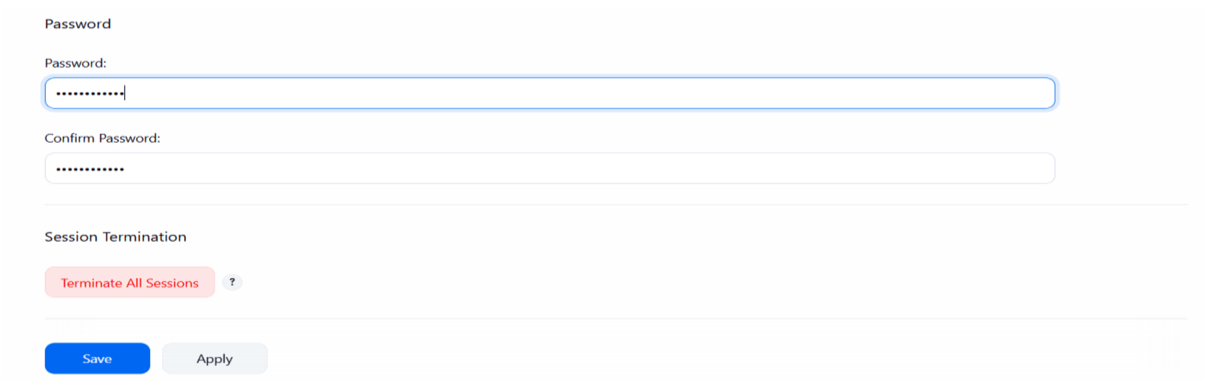
Setup Jenkins is done



2. Secure Jenkins server

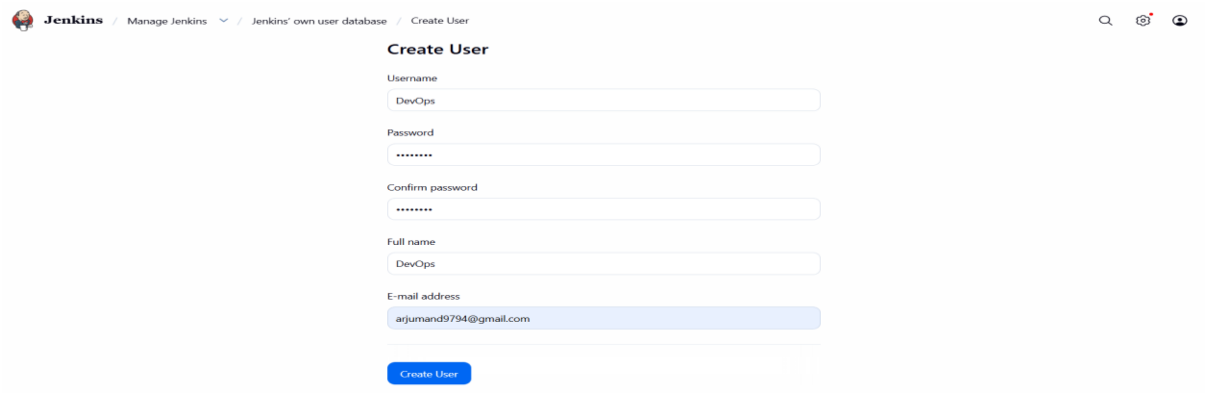
Steps:

- I went to username “Arjumand Logo”
- Selected security
- Then changed the password, initially it was given as default.



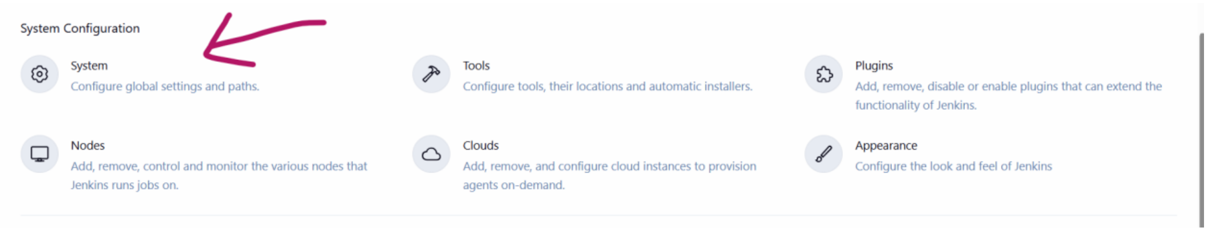
3. Create users called DevOps, Testing in Jenkins with Limited access.

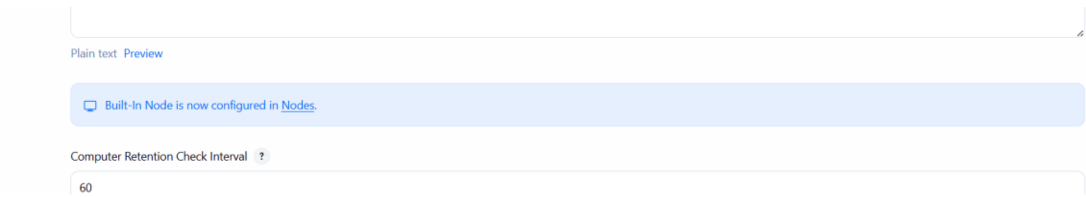
Created an user called DevOps went to manage jenkins, create user and given credentials



User got limited access and created user in manage Jenkins after loggedin to user

4. Configure labels and restrict the jobs to execute based on label only.





Restore Required Backup Data

- Copied only the job directories from the backup to Jenkins home:

sudo mv /home/ec2-user/jenkins_backup/jobs/* /var/lib/jenkins/jobs/

---

1. Apply Correct Ownership & Permissions

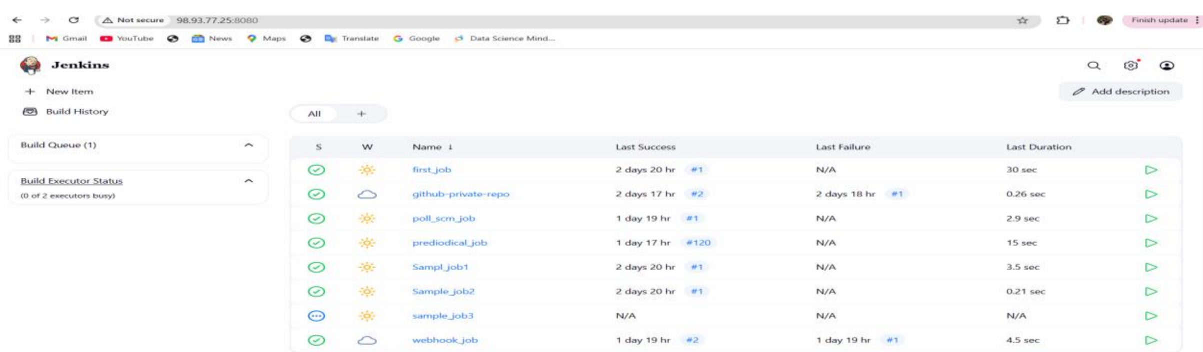
sudo chown -R jenkins:jenkins /var/lib/jenkins

sudo chmod -R 755 /var/lib/jenkins

---

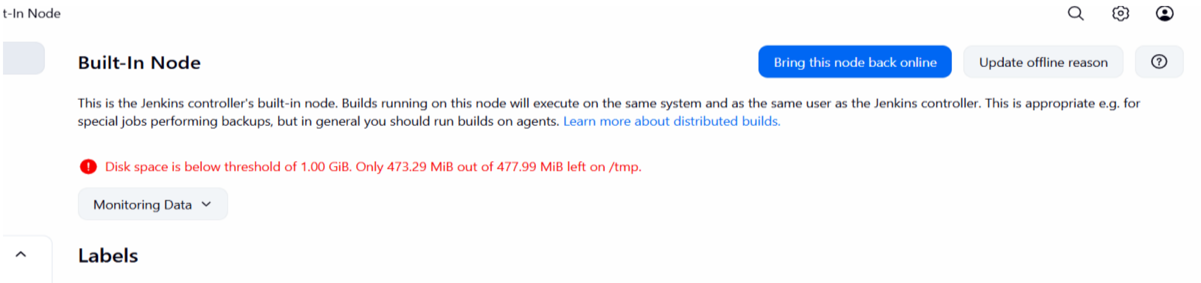
2. Start Jenkins Again

sudo systemctl start jenkins



ERROR:

While I was creating master label I was facing issue related to storage,



Steps followed for troubleshoot:

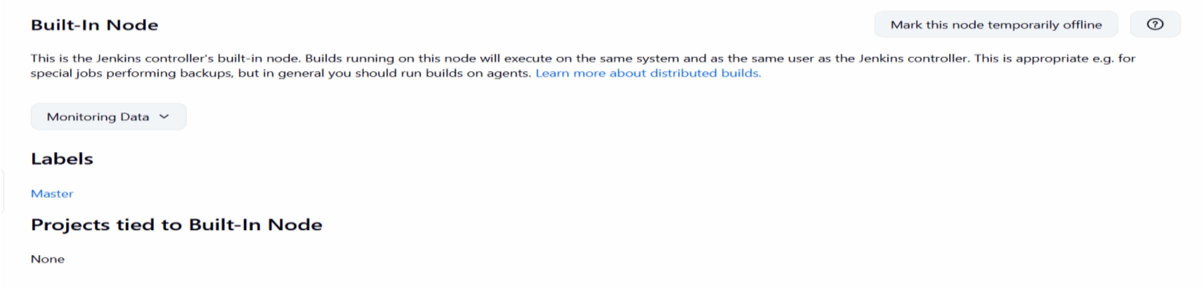
- Identified that the Jenkins built-in node went offline due to **low disk space threshold on /tmp**.
- Increased the EC2 **root EBS volume from 8 GB to 30 GB** and verified the filesystem size.
- Discovered /tmp was mounted as **tmpfs (RAM-based)**, causing Jenkins to still detect insufficient space.
- Redirected Jenkins temporary files to a **disk-based directory** using a systemd override and restarted Jenkins.
- Brought the node back online, verified node health, and confirmed the **label was created and usable**.

Here are 5 concise troubleshooting lines, including EBS modification and the key commands you used, ready for documentation:

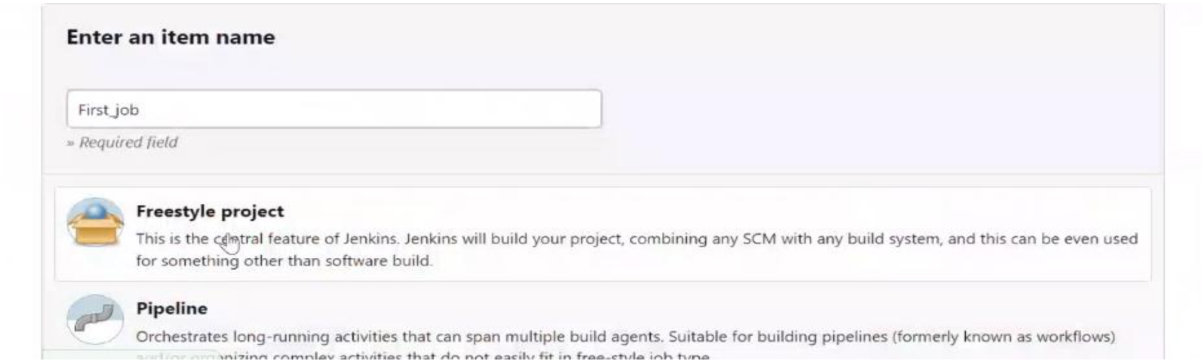
- Detected Jenkins node offline due to low disk space on /tmp and verified usage using
- df -h /tmp
- Modified the EC2 **root EBS volume from 8 GB to 30 GB** via AWS Console and confirmed disk layout with
- lsblk
- Verified that /tmp was mounted as **tmpfs**, explaining the persistent disk warning:
- df -h
- Redirected Jenkins temporary files to disk by updating the systemd override and applied changes:
- sudo vi /etc/systemd/system/jenkins.service.d/override.conf
- sudo systemctl daemon-reload
- sudo systemctl restart jenkins

➢ Brought the Jenkins built-in node back online and validated successful recovery using the Jenkins UI.

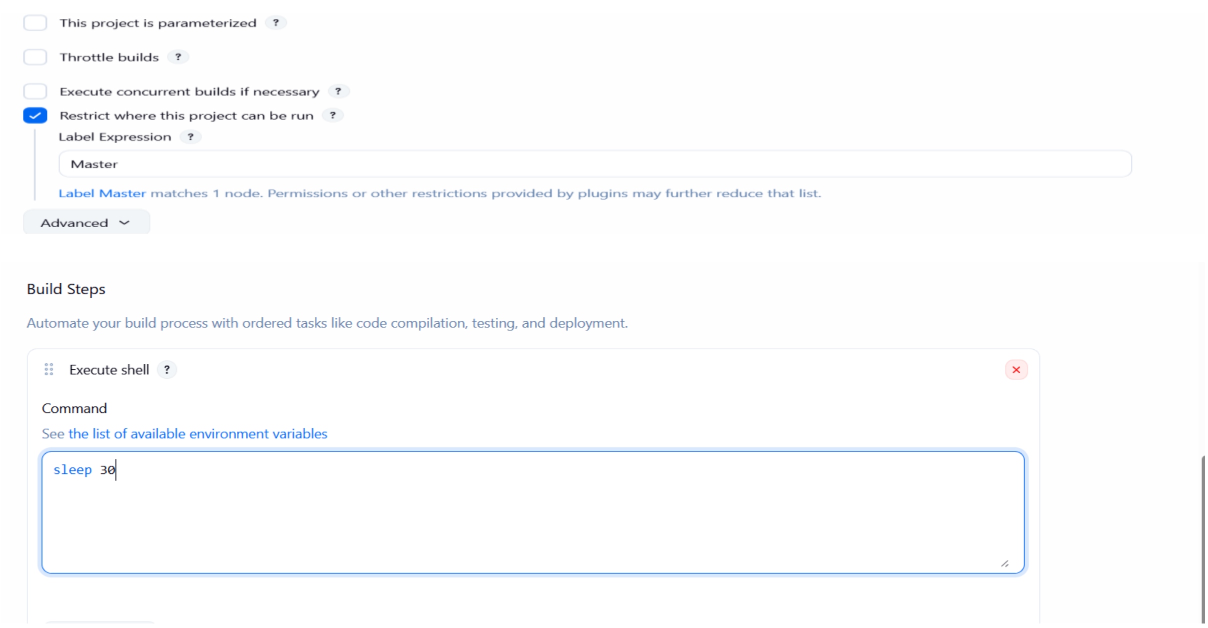
Created label



Created first job in freestyle project



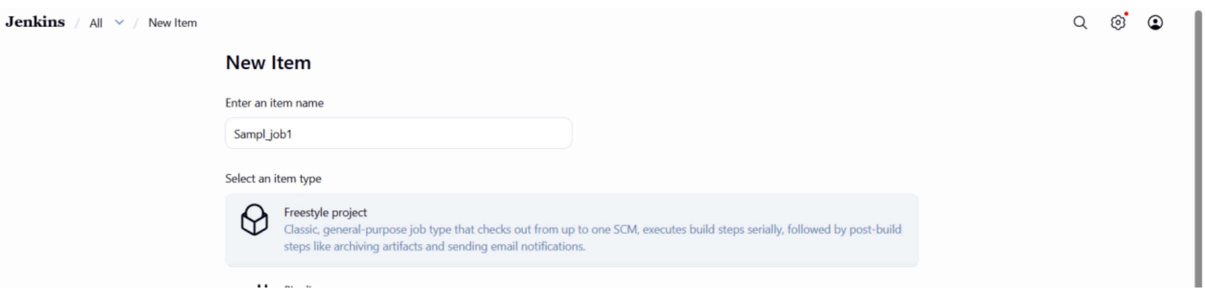
Selected option called restrict where can be run

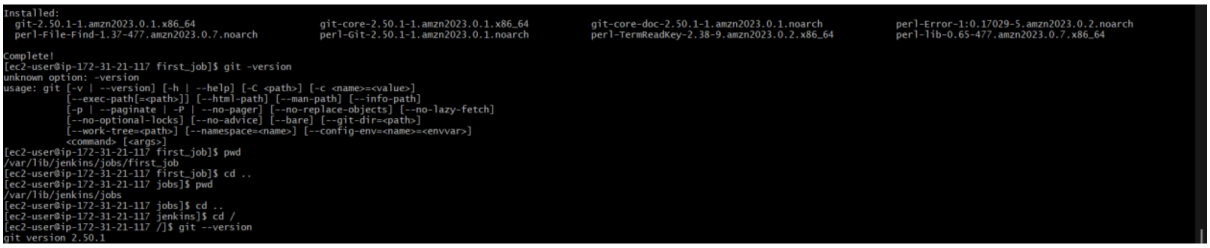


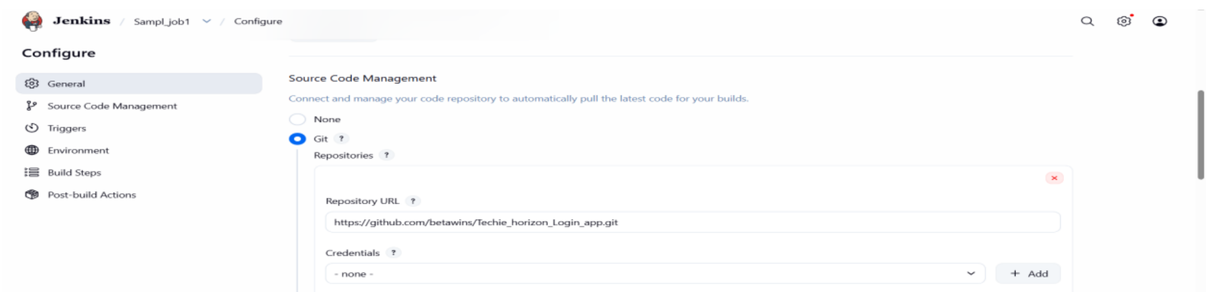
Now job will be executed only in Master label

Job is executed successfully

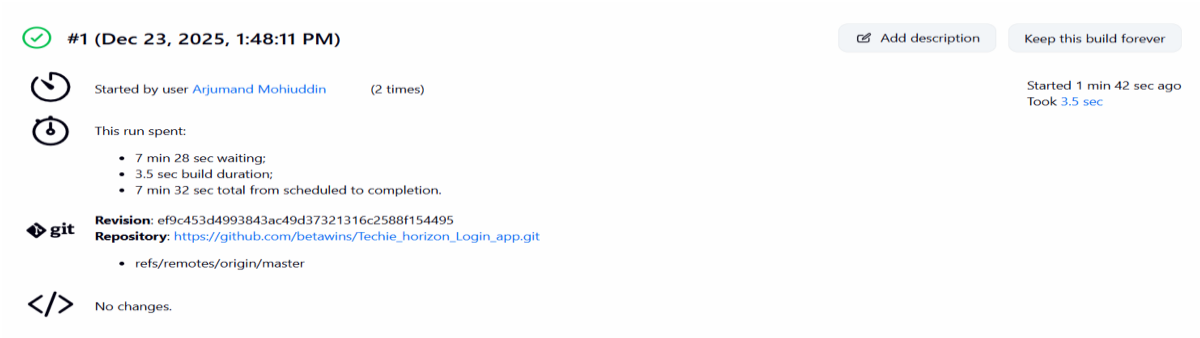
5. Create Three sample jobs using the below URL. https://github.com/betawins/Techie_horizon_Login_app.git



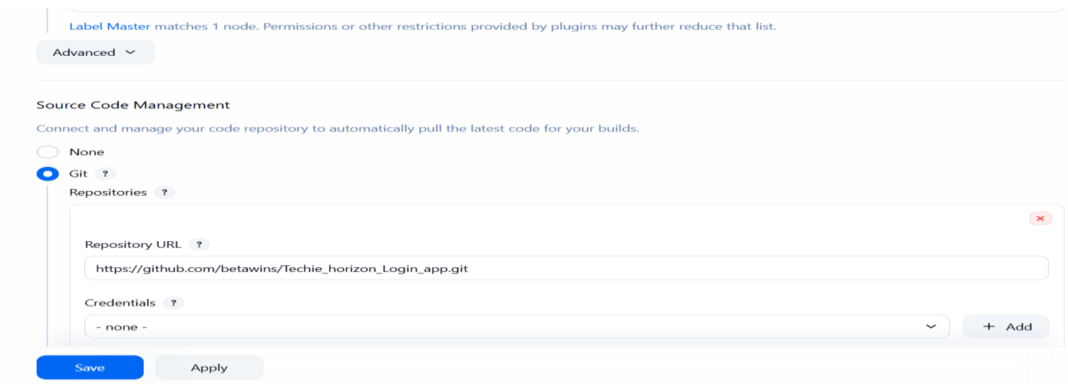


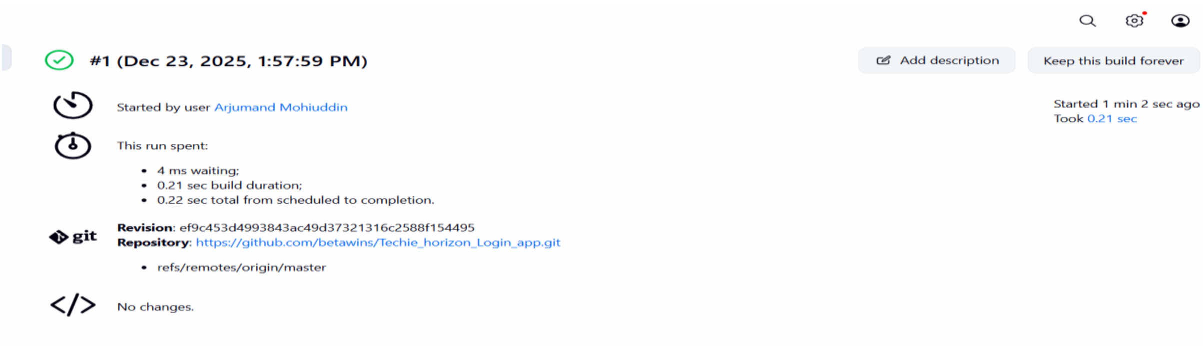


Job is executed sample job 1 using URL without Master label

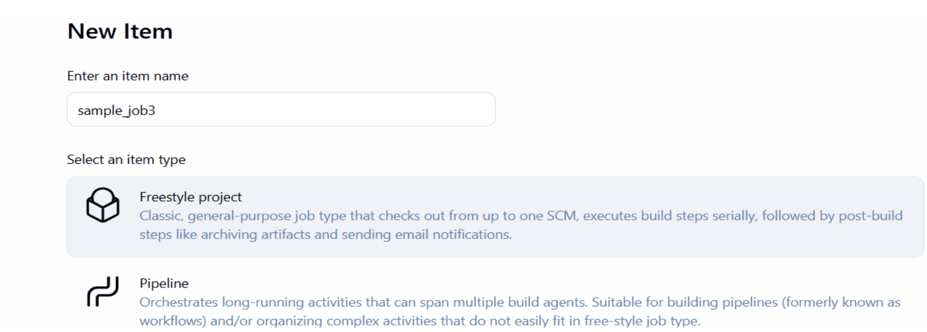


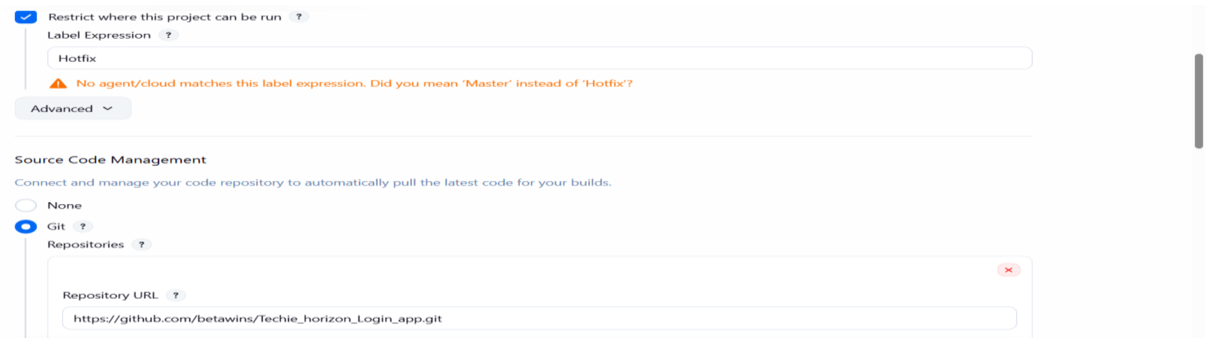
Second sample job using URL with Master label and confirm with status



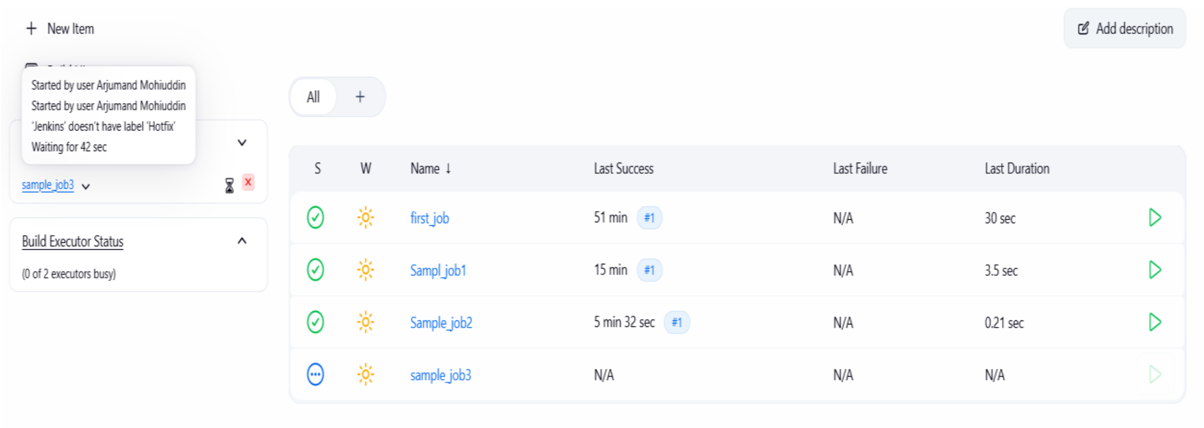


New job with different label hotfix which is not there





As you can see my third sample is not executed yet because that label is not there

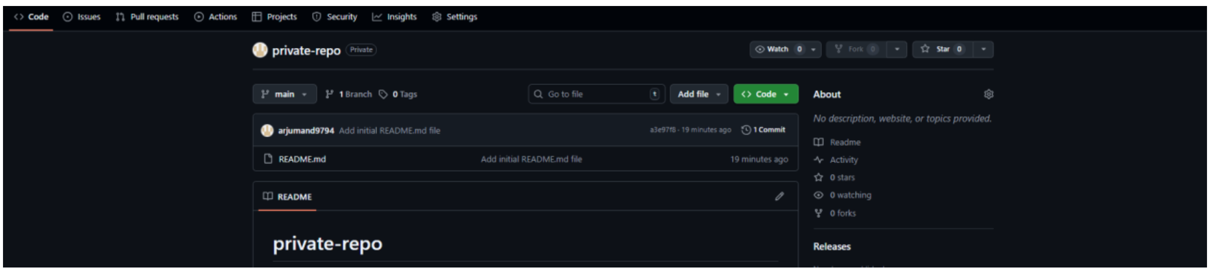


6. Create one Jenkins job using git hub Private repository.

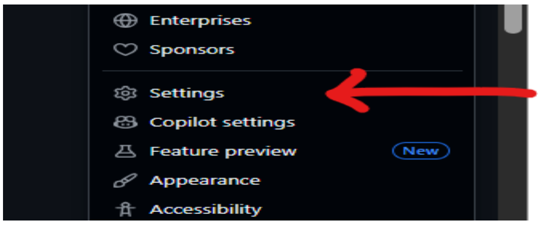
Steps:

1. Create a Private Repository in GitHub

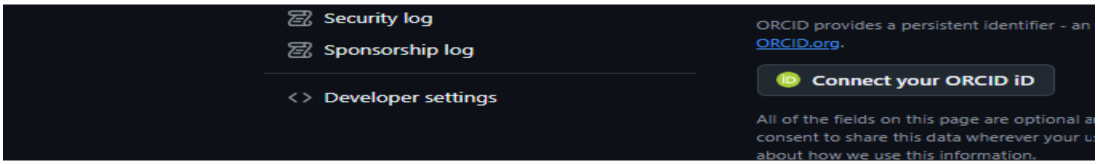
- Logged in to GitHub.
- Created a **private repository** named private-repo.
- Added a README.md file (initial commit).
- Noted the **default branch name: main**.
- Copied the **HTTPS repository URL**:
- https://github.com/arjumand9794/private-repo.git



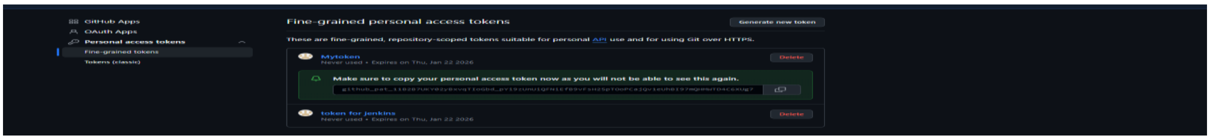
- Go to logo and the settings



- Go to developer setting



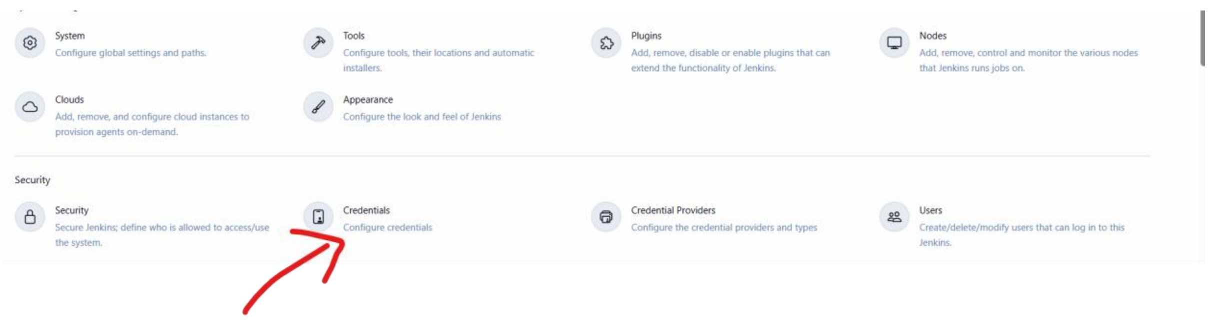
- Create a token classic



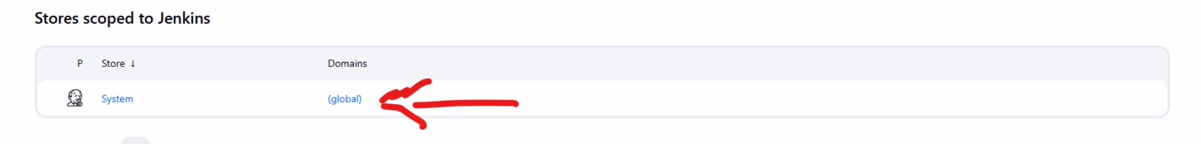
2. Create GitHub Credentials in Jenkins

- Went to Jenkins Dashboard → Manage Jenkins → Credentials.
- Selected (global) → Add Credentials.
- Chose:
  - Kind: Username with password
  - Username: GitHub username
  - Password: GitHub Personal Access Token (PAT)
  - ID: github-private-repo (simple name, no URL)
  - Description: GitHub private repository access
- Saved the credentials.

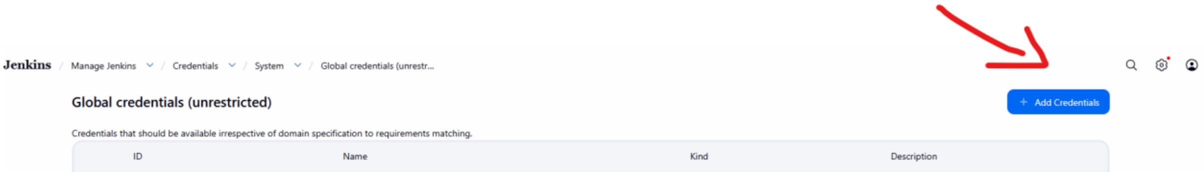
- Go to credentials



- Go to global



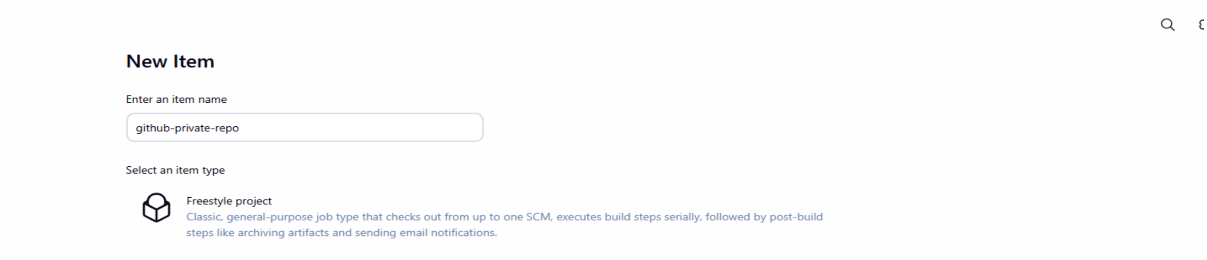
- Go to add credentials



Add user name, password (token) Id then add discrption and then save

3. Create a New Jenkins Job

- Clicked New Item in Jenkins.
- Entered job name: github-private-repo.
- Selected Freestyle project.
- Clicked OK.



Configure Git Repository in the Job

- Opened Configure for the job.
- Under Source Code Management, selected Git.
- Pasted the private repository URL:
- https://github.com/arjumand9794/private-repo.git
- Selected the saved GitHub credentials.

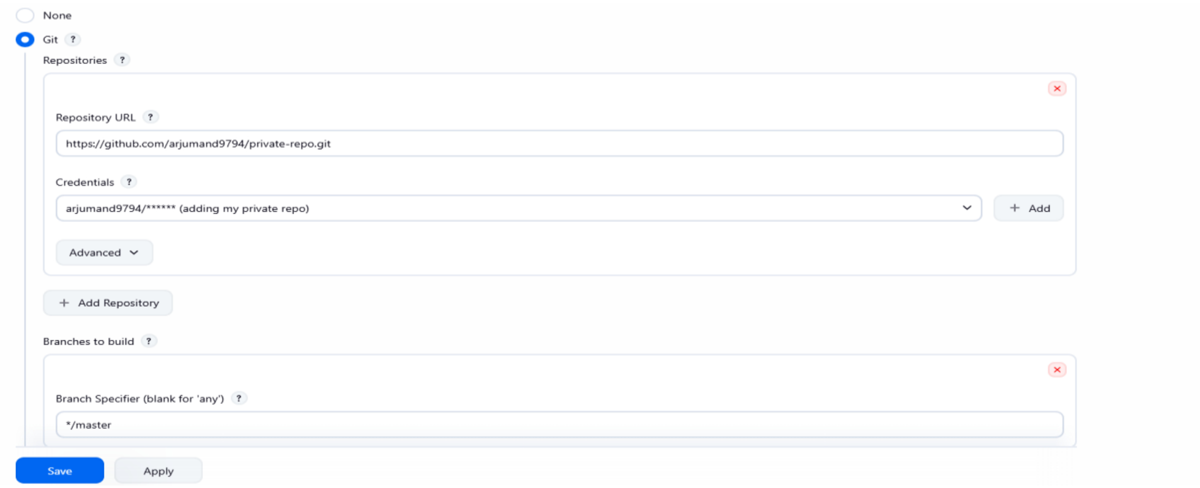
5. Configure Correct Branch (IMPORTANT STEP)

- Under **Branches to build**, set:
- */main
- This was required because the repository uses **main**, not master.

---

6. Save and Build the Job

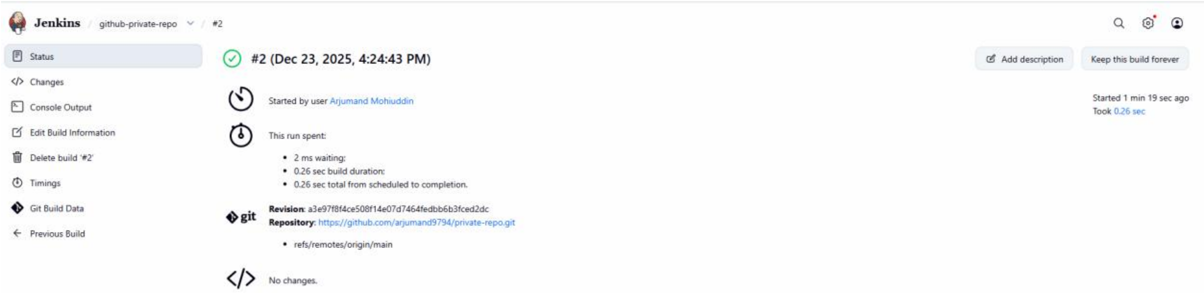
- Clicked **Save**.
- Clicked **Build Now**.



Verify Build Execution

- Opened Build #2.
- Checked:
  - Status: Green tick (SUCCESS)
  - Console Output: No errors
  - Git Build Data:
    - Branch: refs/remotes/origin/main
    - Commit ID displayed
- Confirmed the job executed successfully.

✅ Final Result



Jenkins
github-private-repo
#2
Status
Changes
Console Output
Edit Build Information
Delete build '#2'
Timings
Git Build Data
Previous Build
#2 (Dec 23, 2025, 4:24:43 PM)
Add description
Keep this build forever
Started by user Arjumand Mohiuddin
Started 1 min 19 sec ago
Took 0.26 sec
This run spent:
2 ms waiting;
0.26 sec build duration;
0.26 sec total from scheduled to completion.
git
Revision: a3e97f8f4ce508f14e07d7464fedbb6b3fced2dc
Repository: https://github.com/arjumand9794/private-repo.git
refs/remotes/origin/main
No changes.